

SYSTEM DESIGN INTERVIEW

Design Swiggy / Deliveroo

three-sided food-delivery marketplace: customers, restaurants, riders

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · The Three Lifecycles
- 06 · Database Schema
- 07 · Restaurant Menu Sync
- 08 · ETA Composition
- 09 · Rider Dispatch & Batching
- 10 · Surge & Busy-Area Pricing

- [11](#) · Search & Discovery
- [12](#) · Comparison: Swiggy vs Uber vs DoorDash
- [13](#) · Failure Modes
- [14](#) · Design Trade-offs
- [15](#) · Interview Playbook

01

Problem Statement

Three-sided marketplace, not just rideshare

Design a food-delivery platform like **Swiggy**, **Deliveroo**, **DoorDash**, or **Uber Eats**. Unlike Uber rideshare (two-sided: rider + driver), this is a **three-sided marketplace**: *customer* places an order, *restaurant* prepares the food, and *rider* delivers it. All three lifecycles must stay synchronized on a single `order_id` state machine while an ML stack predicts prep-time and ETA, and the dispatcher batches deliveries for unit economics.

Why This Differs from Uber Rideshare

Dimension	Uber Rideshare	Food Delivery
Sides	2 (rider, driver)	3 (customer, restaurant, rider)
Service time	Known once trip starts	Variable prep_time (5–25 min, ML estimate)
Inventory	Driver capacity = 1 trip	Restaurant menu, in-stock flags, kitchen load
Batching	UberPool: shared mid-trip	Stack 2–3 orders from same restaurant cluster
Cancellation	Either side, low cost	Food already cooking → restaurant absorbs cost
Dispatch trigger	Rider request immediately	Wait for prep-time minus rider arrival
Search	Pickup location only	Restaurant catalog, dish, cuisine, price, rating

Clarifying Questions

Question	Answer
Order volume?	~5M orders/day globally
Cities & restaurants?	~500K active restaurants across major metros
Rider fleet?	~500K riders concurrently online at peak
Search load?	~50K QPS (catalog browsing dominates)
Order placement peak?	~60 orders/sec (lunch + dinner spikes 5–8× baseline)
Geography?	Hyperlocal: most deliveries within 4 km / 30 min

Question	Answer
Payments?	Card, UPI, wallets, cash-on-delivery
Batching allowed?	Yes: 2–3 orders if same direction & overlapping prep

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Browse / Search	Customer searches restaurants by location, cuisine, dish, rating, price
Cart & Checkout	Add items; re-validate prices & stock at checkout (menu may have changed)
Place Order	Atomic order creation: payment hold + restaurant confirmation
Restaurant App	Tablet at restaurant accepts/rejects, marks prep_started / food_ready
Dispatch	Match order to rider considering prep-time, distance, and batch potential
Live Tracking	Customer sees rider location; ETA recomputed continuously
Payments	Pre-auth at order, capture on delivery; refunds for cancellations
Surge Pricing	Busy-area delivery fee multiplier when demand >> supply
Notifications	Push/SMS at each state transition

Non-Functional Requirements

Requirement	Target
Availability	99.95% for order/dispatch; 99.9% for search
Latency (search)	<200ms p95 from edge
Latency (place order)	<1s p95 (payment hold + restaurant push)
Dispatch decision	<3s from food_ready to rider_assigned
Menu propagation	Restaurant edits visible in search index in <60s
ETA accuracy	MAE <3 minutes; p90 within 5 min
Consistency	Strong on order state & payment; eventual on search index

Requirement	Target
Geo coverage	Multi-region active-active per metro

03

Capacity Estimation

Math for scale

Traffic & Concurrency

- Active users (MAU): **~200M**; DAU ~30M
- Orders/day: **~5M** globally
- Average orders/sec: $5M / 86,400 \approx \mathbf{58/sec}$ baseline
- Lunch + dinner peak: $5-8\times$ baseline $\approx \mathbf{\sim 60 orders/sec sustained at peak}$ (with bursts to $\sim 300/sec$)
- Search QPS: customers browse 8–10 screens before ordering $\rightarrow \mathbf{\sim 50K search QPS}$ at peak
- Active riders online: **~500K**; GPS pings every 4s $\rightarrow \mathbf{\sim 125K pings/sec}$
- Active restaurants: **~500K**; tablet poll/long-poll every 5s $\rightarrow \mathbf{\sim 100K conn}$

Storage Estimation

- Per order: header (~ 2 KB) + items (~ 3 KB) + state log (~ 1 KB) $\approx \mathbf{\sim 6 KB}$
- Daily order data: $5M \times 6 KB = \mathbf{\sim 30 GB/day}$
- 5-year retention (hot 90d, warm 1y, cold archive): **~55 TB**
- Restaurant menus: $500K \times \sim 50 KB = \mathbf{\sim 25 GB}$ (small, fits in cache)
- Rider GPS history (90 days): $500K riders \times 1 ping/4s \times 86,400/4 \times 90 \approx \mathbf{\sim 1B pings/day} \rightarrow \mathbf{\sim 120 GB/day}$ compressed
- Search index (Elasticsearch): denormalised restaurant + dish docs $\approx \mathbf{\sim 80 GB}$ per region

Cache & Bandwidth

- Hot menus (top 10% of restaurants drive 80% of orders) $\rightarrow \mathbf{\sim 5 GB Redis}$ per region
- Geo-index of riders (H3 hex cells) $\rightarrow \mathbf{\sim 200 MB}$ in memory per metro
- Customer cart sessions (Redis, TTL 30 min) $\rightarrow \mathbf{\sim 2 GB}$
- Bandwidth: rider pings $100B \times 125K/sec = \mathbf{\sim 12 MB/sec}$ ingress

KEY

Key Numbers

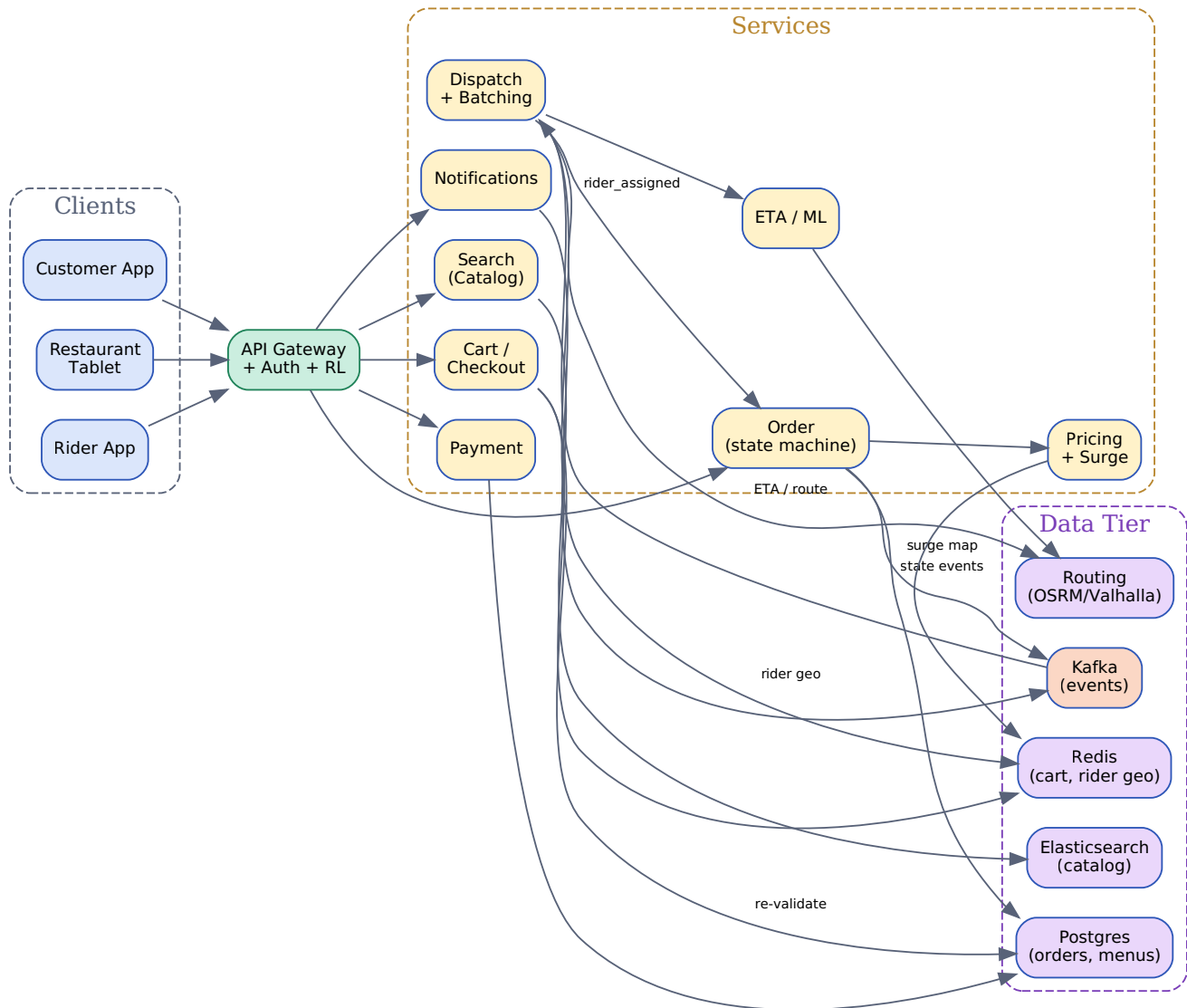
5M orders/day · ~ 60 orders/sec peak · 50K search QPS · 500K riders · 500K restaurants · 30 GB orders/day · MAE ETA <3 min.

04

High-Level Architecture

Three sides into one platform

Three client surfaces — **customer app**, **restaurant tablet**, and **rider app** — all enter through an API gateway and converge on a set of services that share an `order_id` state machine. Search and Cart are read-heavy and run off Elasticsearch + Redis; Order and Dispatch are write-heavy and run off Postgres + Kafka with Redis geo-sets for live rider locations.



Three clients converge on a shared API gateway, fan out to domain services, and persist to a heterogeneous data tier (Postgres for transactions, Redis for hot state, Elasticsearch for catalog, Kafka for events).

Service Responsibilities

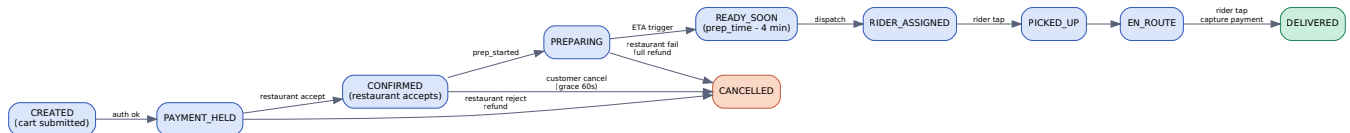
- **API Gateway:** auth (JWT), rate limit, route to the correct service per client kind
- **Search:** Elasticsearch over denormalised restaurant+dish docs, geo-filtered
- **Cart:** Redis-backed session; re-validates price & stock against Postgres at checkout
- **Order:** owns the `order_id` state machine; only writer to `orders` table
- **Dispatch:** consumes `food_ready_soon` events; assigns & batches riders
- **Payment:** pre-auth at `place_order`; capture on delivered; refund on cancel
- **Pricing/Surge:** publishes per-hex multiplier; Cart & Search read it
- **ETA / ML:** separate predictors for prep, arrival, `cooking_buffer`, delivery
- **Notifications:** Kafka consumer → push/SMS/email per state transition

05

The Three Lifecycles

Customer order, restaurant prep, rider dispatch — one state machine

The defining property of food delivery is that three independent actors drive state transitions on the *same* `order_id`. The Order service is the single source of truth; every other service subscribes to the event stream and reacts. State changes go through Postgres with a row-level lock so concurrent updates from rider, restaurant, and customer cannot corrupt the order.



Order state machine. *CREATED* → *CONFIRMED* is the commit point (payment held + restaurant accepted). *DELIVERED* is terminal-success; *CANCELLED* is terminal-failure.

Synchronization Rules

- **Single writer:** only Order service writes to orders; everyone else publishes intent via Kafka
- **Row lock:** `SELECT ... FOR UPDATE` on the order row before transition; reject illegal transitions in SQL CHECK
- **Idempotency:** every event carries (`order_id`, `transition`, `occurred_at`); duplicates are no-ops
- **Forward-only:** states form a DAG; no rollbacks (failed pickup → *CANCELLED*, not back to *CONFIRMED*)
- **Customer cancel grace:** allowed for 60s after *CONFIRMED*; after that, only restaurant or system can cancel

06

Database Schema

Orders, items, and state transitions

Core Tables

```
-- Postgres: order header (one row per order)
CREATE TABLE orders (
  order_id      UUID PRIMARY KEY,
  customer_id   BIGINT NOT NULL,
  restaurant_id BIGINT NOT NULL,
  rider_id      BIGINT NULL,           -- assigned later
  status        VARCHAR(20) NOT NULL,  -- enum: CREATED..DELIVERED
  subtotal_cents INT NOT NULL,
  delivery_fee  INT NOT NULL,
  surge_mult    NUMERIC(4,2) DEFAULT 1.00,
  payment_id    UUID,
  prep_eta_sec  INT,                  -- ML estimate at order time
  delivery_eta_sec INT,
  pickup_h3     BIGINT,               -- H3 hex of restaurant
  drop_h3       BIGINT,               -- H3 hex of customer
  created_at    TIMESTAMP DEFAULT NOW(),
  updated_at    TIMESTAMP DEFAULT NOW(),
  CHECK (status IN ('CREATED', 'PAYMENT_HELD', 'CONFIRMED',
                    'PREPARING', 'READY_SOON', 'RIDER_ASSIGNED',
                    'PICKED_UP', 'EN_ROUTE', 'DELIVERED', 'CANCELLED'))
);
CREATE INDEX idx_orders_rider      ON orders(rider_id, status);
CREATE INDEX idx_orders_rest_open ON orders(restaurant_id, status)
  WHERE status NOT IN ('DELIVERED', 'CANCELLED');

-- Items (line items per order)
CREATE TABLE order_items (
  order_id  UUID REFERENCES orders(order_id),
  line_no   INT,
  dish_id   BIGINT NOT NULL,
  name_snap TEXT NOT NULL,           -- snapshot of name at order time
  price_snap INT NOT NULL,           -- snapshot of price at order time
  qty       INT NOT NULL,
  options   JSONB,                   -- size, addons, no-onion, etc.
  PRIMARY KEY (order_id, line_no)
);

-- Append-only state transition log (audit + event sourcing)
CREATE TABLE order_events (
  event_id  BIGSERIAL PRIMARY KEY,
  order_id  UUID NOT NULL,
```

```

from_status VARCHAR(20),
to_status   VARCHAR(20) NOT NULL,
actor       VARCHAR(20),           -- customer|restaurant|rider|system
reason      TEXT,
occurred_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_oe_order ON order_events(order_id, occurred_at);

```

Atomic State Transition

```

-- Single transaction: lock + check + write + emit event
BEGIN;
SELECT status FROM orders WHERE order_id = :id FOR UPDATE;
-- application asserts allowed_transition(current, :next)
UPDATE orders
  SET status = :next, updated_at = NOW()
  WHERE order_id = :id
     AND status = :expected_current;    -- optimistic guard
INSERT INTO order_events(order_id, from_status, to_status, actor, reason)
VALUES (:id, :expected_current, :next, :actor, :reason);
COMMIT;
-- Outbox row picked up by Debezium → Kafka topic `order.events`.

```

Sharding & Partitioning

- **Partition orders** by created_at (monthly) — old months drop fast, hot month is small
- **Shard by customer_id hash** across 16 Postgres shards (Citius or app-level) for write scale
- **Co-locate order_items** on the same shard as orders via the same shard key
- **Read replicas** for analytics & restaurant dashboards (eventual consistency OK)

07

Restaurant Menu Sync

Inventory and price freshness

The catalog is read >> write but writes are *operationally critical*: a restaurant turning off an item must propagate everywhere within seconds, or customers will order food the kitchen cannot make. This is the equivalent of inventory in e-commerce, but with much lower SKU count and much higher edit rate (in/out of stock toggles all day).

Write Path: Menu Edit

1. Restaurant edits item on tablet: PATCH /menu/items/:id
2. Menu service writes Postgres (source of truth); commits a row in the outbox
3. Debezium / outbox poller publishes to Kafka topic menu.changes
4. **Search indexer** consumes & upserts the dish doc into Elasticsearch (target <60s)
5. **Cache invalidator** deletes Redis keys menu:rest:{id} and per-dish keys
6. **Cart re-validator** scans active carts that contain that dish; flags them stale

Read Path: Menu Browse

- Search: ES query with geo filter (distance <= 6 km) + cuisine + open_now
- Restaurant detail: Redis menu:rest:{id} (TTL 5 min); miss → Postgres + warm
- Dish image & static assets: served from CDN, cache-busted by version hash
- **Stale-while-revalidate** in the customer app: render last menu instantly, then patch on response

Cart Re-validation at Checkout

```
# Pseudocode: revalidate before locking payment
def checkout(cart):
    fresh = db.fetch_dishes([li.dish_id for li in cart.items])
    issues = []
    for li in cart.items:
        d = fresh[li.dish_id]
        if not d.in_stock:
            issues.append(('OUT_OF_STOCK', li.dish_id))
        elif d.price_cents != li.price_snap:
            issues.append(('PRICE_CHANGED', li.dish_id,
                          li.price_snap, d.price_cents))
    if issues:
        return CartReview(issues)          # show user, require ACK
    return place_order(cart)               # snapshot prices into order_items
```

WARNING

Why snapshot prices

Once an order is placed we copy name and price into `order_items` as snapshots. The dish row is mutable; the order is immutable. This is the same pattern Amazon uses: a price change after checkout must never affect a placed order.

08

ETA Composition

Four ML models, not one

What the customer sees as a single "35 min" ETA is actually the sum of four independent estimates, each with its own model and feature set. Decomposing them lets us debug accuracy per stage and recompute only the stale piece.

The Four Components

Component	What it estimates	Key features
prep_time	Restaurant cook + plate time	dish complexity, kitchen load, hour-of-day, restaurant historical p50/p90
rider_arrival	Time for assigned rider to reach restaurant	rider current loc, traffic, mode (bike/scooter), batch leg
cooking_buffer	Slack so rider doesn't wait at restaurant	prep variance, rider on-time risk; usually 1–3 min
delivery_time	Restaurant → customer drop	OSRM route, traffic, building entry penalty, batch order

Composition Formula

```
ETA_total = max(prepare_time, rider_arrival + cooking_buffer)
              + delivery_time

# We display ETA_total to the customer.
# Internally we track each piece so we can recompute lazily:
#   - prep_time updates on PREPARING event
#   - rider_arrival updates on each rider GPS ping
#   - delivery_time updates when rider picks up
#
# Why max(): the rider must wait if they arrive before food is ready;
# the food must wait if it's ready before the rider arrives.
# cooking_buffer biases toward food-waiting (cold-food risk < rider-idle).
```

Dispatch Trigger

- Naive: assign rider at CONFIRMED → rider sits idle 10 min at busy restaurant
- Better: assign at READY_SOON = prep_time - rider_arrival - 1 min

- Trigger fires when $\text{now} \geq \text{confirmed_at} + (\text{prep_time} - \text{predicted_rider_arrival} - \text{buffer})$
- Implementation: scheduled task per order in Redis sorted set keyed by `trigger_at`

INSIGHT

Why a separate ETA service

The ETA service is consulted by Search ("30–35 min delivery"), Cart ("order before 7:45 to get it by 8:15"), Dispatch (when to assign), and the customer tracking screen (live updates). Centralising the model lets us A/B test new estimators in one place.

09

Rider Dispatch & Batching

Greedy nearest-neighbour with prep-time and same-direction constraints

Dispatch is the hardest service. Like Uber it must match supply & demand in a hex grid in real time, but unlike Uber it has two extra constraints: **prep-time** (don't assign too early) and **batching** (stack up to 3 orders to lift rider earnings & platform margin).

Candidate Selection

1. Compute pickup_h3 (H3 res 9, ~150m hex) and search rings 0..3 around it
2. Pull online riders in those hexes from Redis GEORADIUS / H3 set
3. Filter: rider state IN (FREE, BATCH_OPEN), capacity left, vehicle type matches
4. Score each candidate: $\text{cost} = \text{ETA_to_restaurant} + \alpha * \text{detour} + \beta * (1/\text{rider_acceptance})$
5. Pick top-5; offer to #1 with a 12-second timeout; on decline, fall through

Greedy Batching Algorithm

```
# Greedy nearest-neighbour batching with same-direction constraint.
# Called when a new order_0 reaches READY_SOON.
#
# Goal: either (a) start a new solo trip, or (b) graft order_0 onto
# an existing rider's route if it doesn't blow either ETA.

MAX_BATCH      = 3                # never stack more than 3
MAX_DETOUR_SEC = 6 * 60           # 6 min per added stop
BEARING_TOL_DEG = 35              # 'same direction' window
ETA_BREACH_SEC = 4 * 60           # don't make any order > 4min late

def assign(order_0):
    cands = nearby_riders(order_0.pickup_h3, rings=3)
    best, best_cost = None, INF

    for r in cands:
        if r.state == 'FREE':
            cost = solo_cost(r, order_0)
            if cost < best_cost:
                best, best_cost = ('SOLO', r, None), cost
            continue

        if r.state == 'BATCH_OPEN' and len(r.legs) < MAX_BATCH:
            # Same-direction check on bearings
            if bearing_diff(r.next_drop, order_0.drop) > BEARING_TOL_DEG:
```

```

        continue
    # Insert order_0 at best position; recompute legs
    new_legs, detour = best_insertion(r.legs, order_0)
    if detour > MAX_DETOUR_SEC:
        continue
    if any(eta_breach(l) > ETA_BREACH_SEC for l in new_legs):
        continue
    cost = batch_cost(r, new_legs, detour)
    if cost < best_cost:
        best, best_cost = ('BATCH', r, new_legs), cost

if not best:
    return queue_for_retry(order_0)    # back-off, surge area
return offer(best)                   # 12s accept window

```

Batch Eligibility

- **Same restaurant cluster:** pickups within 200m and prep windows overlap
- **Same direction:** drop bearings within 35° of each other
- **Detour budget:** additional stop adds ≤ 6 minutes per existing leg
- **No ETA breach:** no customer's promised time slips by more than 4 minutes
- **Cap:** max 3 orders per rider; cold-food risk grows non-linearly

Dynamic Re-routing

- Re-evaluate the leg order on every rider GPS ping (4s interval)
- If traffic recomputes leg ordering and it doesn't breach ETA, swap silently
- If a leg *will* breach: notify customer + suppress further insertions on that rider
- Mid-trip cancel: drop the leg, refund, recompute remaining route

10

Surge & Busy-Area Pricing

Closing the supply gap

When demand outstrips rider supply in a hex, ETAs blow up and orders stall. Surge does two things at once: it **damps demand** (higher fee filters out price-sensitive orders) and **summons supply** (riders see a heatmap and self-relocate).

Surge Computation

```
# Per H3 hex, every 60 seconds:
open_orders  = count(orders waiting for rider in hex over last 5 min)
free_riders  = count(riders idle/free in hex)
p90_wait_sec = percentile90(time-to-rider_assigned in hex)

demand_pressure = open_orders / max(free_riders, 1)
wait_pressure   = clamp(p90_wait_sec / 180.0, 0, 3)    # 3min target

surge = round(1.0 + 0.4*max(0, demand_pressure-1)
              + 0.3*max(0, wait_pressure-1), 1)
surge = clamp(surge, 1.0, 2.5)                        # cap
```

How It Surfaces

- **Customer:** delivery fee × surge_mult shown *before* they tap Place Order
- **Rider:** heatmap colours hexes by surge; rider self-relocates to earn more
- **Restaurant:** unaffected — restaurant takes their normal cut
- **Storage:** 60s-resolution surge_map kept in Redis, hex → multiplier

WARNING

Surge gotchas

Cap surge (here 2.5×) to avoid PR disasters during emergencies (cap to 1.0× during declared natural disasters). Snapshot surge_mult onto the order at checkout so the price the user saw is the price they're charged.

11

Search & Discovery

Catalog at 50K QPS

Index Structure

- **One Elasticsearch index per metro**; reduces shard count and keeps geo queries local
- **Document = restaurant** with nested dish list (denormalised)
- **Geo field** on restaurant location; geo_distance filter at query time
- **Computed fields**: open_now, avg_prep_time, delivery_eta, rating, price_band

Ranking Signals

Signal	Why
Distance + ETA	Closer = faster & cheaper to deliver
Rating × order count	Bayesian average; avoids one-review outliers
Personalisation	User's past cuisines, repeat orders
Operational health	Cancellation rate & on-time rate downweight bad actors
Sponsored	Paid placement, capped & labelled

Indexing Pipeline

- Postgres (truth) → outbox → Kafka menu . changes
- Indexer service consumes, builds doc, upserts to ES & CDN-cached static menu
- Hot path edits (in_stock toggle): high-priority partition, target latency <30s
- Cold path edits (dish photo, description): low-priority, <5 min OK

12

Comparison: Swiggy vs Uber vs DoorDash

And batch-1 vs batch-2 economics

Platform Comparison

Aspect	Uber Rideshare	Swiggy / Deliveroo	DoorDash
Sides	2	3 (customer/restaurant/rider)	3 (same)
Trip time predictability	Known after pickup	Variable prep_time + traffic	Same as Swiggy
Batching strategy	UberPool: shared mid-route	Stack 2–3 orders, same-direction	DoubleDash: 2 orders/trip
Surge unit	Per hex, immediate	Per hex, demand+wait pressure	Peak Pay (per zone, scheduled)
Avg delivery radius	5–25 km	<4 km hyperlocal	~4 km
Restaurant integration	n/a	Tablet POS push	Mostly tablet; some POS
Rider mode	Car	Bike/scooter dominant	Car & bike
Order size	1 person	1–4 people, multi-item	Same

Batch-1 vs Batch-2 Unit Economics

Metric	Solo (batch=1)	Stacked (batch=2)
Avg trip time	26 min	32 min
Trips/hour	2.3	1.9 (but 3.8 deliveries)
Deliveries/hour	2.3	3.8
Rider revenue/hour	\$15	\$22 (+47%)
Platform CoG/order	\$5.20	\$3.40 (-35%)
Customer ETA delta	0	+4 min on second drop

Metric	Solo (batch=1)	Stacked (batch=2)
Cold-food risk	Low	Medium (food sits during 1st drop)

KEY

Why batching matters

Batching is the single biggest lever on food-delivery margin. Going from 100% solo to 30% stacked drops cost-per-order by ~10%, which is the difference between a profitable and unprofitable city. The constraint is customer trust — push batching too hard and ETAs slip, NPS craters.

13

Failure Modes

What goes wrong, and how to recover

Failure	Impact	Detection	Mitigation
Restaurant tablet offline	Orders queue without acceptance	Heartbeat > 30s lost	SMS fallback to manager; auto-cancel + refund after 90s
Rider GPS gap	Stale dispatch decisions; bad ETA	No ping > 30s	Dead-reckon from last vector; freeze rider state; re-dispatch if > 2 min
Payment provider down	Cannot pre-auth → orders fail at checkout	Provider error rate > 5%	Failover to secondary PSP; allow COD as fallback
Postgres primary down	Order writes block	Replication lag spike, write timeout	Auto-promote replica (Patroni); 30–60s blackout
Elasticsearch lag	Stale catalog, in-stock toggles invisible	Indexer lag > 60s	Reduce concurrency on cold edits, scale indexer; show 'temp unavailable' on stale items
Routing service (OSRM) slow	ETA inaccuracy, dispatch delays	p99 latency > 1s	Fall back to haversine × city factor; degrade ETA precision
Surge runaway	Customer outrage, PR risk	Surge > 2.5× sustained	Hard cap; manual kill-switch per metro
Rider mass cancel	Orders pile up, prep waste	Acceptance rate drops > 30%	Trigger surge + ops bonus; throttle new orders in hex
Kafka lag	Notifications & dispatch delayed	Consumer lag > 30s	Add partitions; spill to dead-letter; replay

WARNING

Cooking-cost asymmetry

Once an order is **CONFIRMED**, the restaurant has spent ingredients and labour. Any cancel after **CONFIRMED** has a real cash cost. Design the cancel path so it's cheap to cancel *before* **CONFIRMED** (60s customer grace) and expensive after (only restaurant or system, with compensation policy).

14

Design Trade-offs

What we picked and why

Decision	Choice	Trade-off
Order state authority	Single Order service	Bottleneck risk; but eliminates split-brain across rider/restaurant/customer
Menu freshness	Eventual (<60s) in search; strong at checkout	Customers may see stale availability briefly; cart re-validation catches it
Price snapshot	Snapshot at order_items write	Price changes can't affect placed orders; menu edits are safe
Dispatch trigger	READY_SOON (prep_time - rider_arrival)	Rider doesn't sit idle; risks late dispatch if prep prediction is wrong
Batching cap	3 orders max	Bigger batches earn more but cold-food & ETA-breach risk grows non-linearly
Same-direction window	35° bearing tolerance	Tighter = fewer batches; looser = bigger detours
ETA model	4 separate predictors	More moving parts; but each component is debuggable + replaceable
Surge cap	2.5×	Lost revenue at extreme demand; protects brand & regulator scrutiny
Datastore mix	Postgres + Redis + ES + Kafka	Operational complexity; but each tool fits one read pattern
Geo index	H3 (Uber's lib)	Hex cells distort less than lat/lon grid; ring queries cheap

TIP

Pick your hill

The two design hills you should die on in an interview: (1) **order state machine is single-writer**, all sides publish intents; (2) **batching is the margin lever**, everything in dispatch (geo, ETA, surge) exists to make safe batching possible.

15

Interview Playbook

How to present this in 45 minutes

If you've already discussed Uber rideshare, the interviewer almost always asks for food delivery as a follow-up. They want to see you *not* reuse the rideshare answer wholesale. Anchor on the three differences: third side (restaurant), variable service time (prep), and batching.

45-Minute Outline

1. **Intro (3 min):** three sides, ~5M orders/day, ~60 orders/sec peak, ~50K search QPS
2. **Lifecycle (5 min):** draw the order state machine; emphasise single-writer Order service
3. **High-level arch (5 min):** three clients → API GW → 8 services → Postgres + Redis + ES + Kafka
4. **Schema (5 min):** orders, order_items (price snapshot!), order_events (audit log)
5. **Menu sync (5 min):** <60s search index lag, cart re-validation at checkout
6. **ETA composition (5 min):** four separate models; $\max(\text{prep}, \text{rider_arr} + \text{buffer}) + \text{delivery}$
7. **Dispatch + batching (8 min):** H3 hex → candidates → score → greedy batch insertion
8. **Surge (3 min):** per-hex multiplier, capped, snapshot on order
9. **Failures + trade-offs (5 min):** tablet offline, payment outage, batching cap

Talking Points That Score

- “Three sides, one order_id state machine, single writer.” — shows you understand consistency boundary
- “Snapshot price into order_items.” — shows you've thought about menu mutability
- “ETA = $\max(\text{prep}, \text{rider_arrival} + \text{buffer}) + \text{delivery}$.” — shows you understand the synchronisation
- “Dispatch fires on READY_SOON, not CONFIRMED.” — shows you know rider-idle is a real cost
- “Batch cap 3, bearing tolerance 35°, detour ≤ 6 min.” — concrete numbers beat hand-waving
- “Surge capped at 2.5×, snapshot on order.” — shows you've thought about brand & legal risk
- “H3 hex grid for geo.” — same vocabulary as Uber Eng

Common Follow-ups

- **Q: Restaurant rejects after CONFIRMED — what happens?** A: System cancel + full refund + chargeback to restaurant; reliability score drops; repeat offenders auto-paused.
- **Q: Customer wants to add an item after placing the order.** A: Treat as new mini-order to same restaurant; new payment, new dispatch decision (often piggybacks on same rider).
- **Q: How do you stop riders from gaming surge?** A: Detect cluster of accept-then-cancel; require min on-time rate; pay surge as bonus on completion, not on accept.

- **Q: Cold food complaints?** A: Track temperature-proxy = elapsed since food_ready; cap batches when proxy > 8 min; refund automatically.
- **Q: How is this different from Uber?** A: Three sides not two; variable service time (prep); batching across pickups; restaurant inventory acts as a third consistency boundary.

KEY

Memorize these numbers

5M orders/day · ~60 orders/sec peak · 50K search QPS · 500K riders · 500K restaurants · ETA MAE <3 min · menu propagation <60s · batch cap 3 · surge cap 2.5×